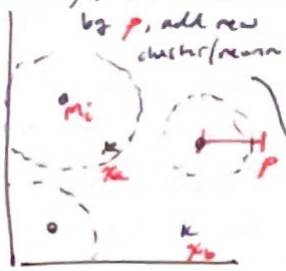
Adaptive Resonance Theory

↳ main intuition: interaction between top-down expectations w/ bottom-up sensory information

↳ new input data
↳ if not swallowed by ρ, add new cluster/neuron



→ comparison prior to updating weights based on vigilance parameter ρ

→ otherwise, update weights for winning neuron towards input

→ k recognition neurons/clusters

Self-Organizing Maps

* competitive learning rate: update centers

$$m \leftarrow m + \eta(x - m)$$

weight

↳ 3 principles: competition, cooperation, adaptation

each neuron in lattice calculates a discriminant function

↳ neurons w/in neighborhood can update

↳ winner + neighbors increase discriminant value relative to input

redundancy is necessary

↳ showing data twice is good

↳ Performance is gauged by Quantization & Topographic

↳ avg d between inputs & BMUs

↳ proportion of inputs where BMU1 & BMU2 not adjacent

Algorithm:

1. initialize weights $w \in W$
2. sample input x
3. find BMU: $i(x) = \arg\min_j \|x - w_j\|$
4. update weights $w_j \leftarrow w_j + \eta h(j, i(x))(x - w_j)$

learning rate

neighborhood function

$\arg(a(x))$

$\arg(\|x_i - w_i(x_i)\|)$

discrete 1 or 0

RBF

→ a layer of RBF ≡ a line of

↳ locally-tuned Taylor terms (approximate subsets of data)

→ output weights can be calculated in one shot (exactly)

when # of inputs = # of RBF units

and approximated when # of inputs > # of RBF units

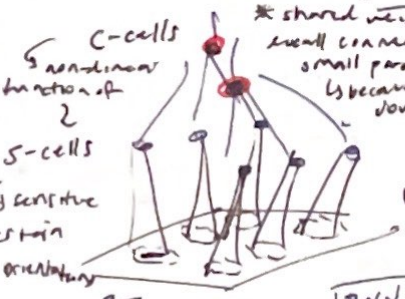
first learn means & stds then use gradient descent to optimize functions at means

$$Fw = y \quad \therefore w = F^{-1}y$$

↳ $w = (F^T F)^{-1} F^T y$

CNNs

* shared weights, small connections, small params
↳ because progressive downsampling



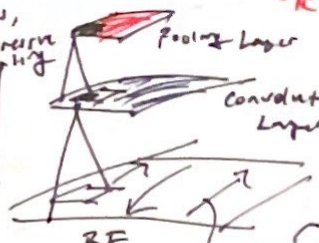
↳ nonlinear function of

↳ sensitive to certain colors, orientations

hierarchy forms from multiple layers;

C-cell outputs feed back to > dimension S-cell map

size of kernel



complexity of features

↳ Kernel/stride → shared by all spatial locations of input

RNNs

BPTT: train by unfolding seq of time in time to steps inputs in FFN & unroll

↳ overcomes vanishing gradient problem (exp. shrinking in deep FFNs)

+ LSTM overcomes memory capacity issue

→ input, forget, and output gates

DQNs

→ delayed/periodic update of target A-V function

↳ record - collect - replay $\langle s_t, a_t, r_t, s_{t+1} \rangle$

↳ input: video frames,
output: $Q(s,a)$ where reward is game score
↳ action-value function

* moving target $\hat{Q}$ used to compute target reward value y_t

$$y_t = \begin{cases} r_t & \text{if prior to termination} \\ r_t + \gamma \arg\max \tilde{Q}(\phi_{t+1}, a'; \theta^-) & \end{cases}$$

→ frozen Q

↳ preprocessed frames/state

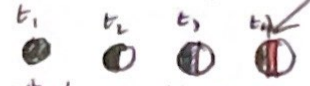
* do a bunch of updates before changing $\hat{Q}$ to ensure/allow proper learning

every C steps, $\hat{Q}$ reset to Q ← gradient descent step on:

$$(y_t - Q(\phi_t, a_t; \theta))^2$$

↳ not moving Q
∴ used to select actions

accounts for current transition, ie new info, to train weights



what is that ?

Decision Tree Learning

— pure induction w/in a completely expressive H

↳ given example pairs $(x, t(x))$ return h that approximates f

↳ Occam's Razor — preference for shallower, accurate trees

↳ a path from root to leaf represents a conjunction of constraints on attribute values for a given outcome

— greedy learning

* Inductive Bias (Occam's R, Inc. search)

* Learning: greedily select attributes based on information gain (reduction in entropy) w/ best attributes at root close to

↳ preference bias vs restriction bias

$$\text{Entropy}(S) = -p_{\oplus} \log_2 p_{\oplus}$$

proportion + examples in S

average "surprisal" of events

sample $-p_{\oplus} \log_2 p_{\oplus}$ — # + examples in S, e.g. — $p_{\oplus} = \frac{n_{\oplus}}{n_{\oplus} + n_{\ominus}}$

* by performing some query yielding S_2 from S_1, and $E(S_1) > E(S_2)$

↳ uncertainty decreases ∴ info gained $(E(S_1) - E(S_2))$

$$\text{Entropy}(S) = \sum_{i \in C} -p_i \log_2 (p_i)$$

set of examples; classes

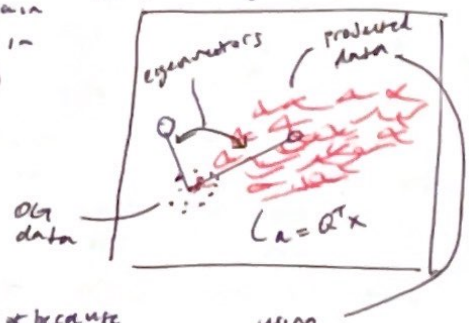
$$\text{IGain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} E(S_v)$$

single attribute; examples where $A = v$

PCA

↳ project a dataset s.t. variance is maximized

↳ center on origin, rotate axes



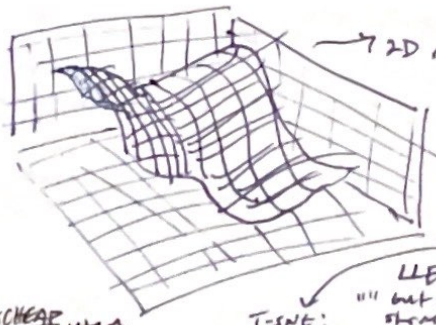
* because Q^T is symmetric, eigenvectors are orthogonal

+ A is INVERTIBLE → can project back to input space ($x = Qa$)

using $Q^T = \begin{bmatrix} 3 & 4 \\ 4 & 3 \end{bmatrix}$

Manifold Learning

→ topological space that is locally euclidean



→ 2D manifold in 3D space — even the surface of a sphere

* task: recover the 2D manifold [2D sphere] from Data

T-SNE: map probability distributions to ↓ dim min. KL divergence

LLE: "but local structure" $P(a_i|v_j) = \frac{n_{\text{along}}}{n_{\text{from}}}$ build graph based on ϵ, see MDS to optimize geodesic distance

Isomap: given pairwise distances, map to ↓ dim s.t. distances are preserved

Bayesian Learning

↳ learning algorithms that deal directly w/ probability

↳ basic thm: given events $A_1, \dots A_n$, mutually exclusive, i.e. $\sum_i P(A_i) = 1$ —

$$P(B) = \sum_1^n P(B|A_i) \cdot P(A_i)$$

↳ Bayes'

$$P(h|D) = \frac{P(D|h) P(h)}{P(D)} = \frac{P(h \cap D)}{P(D)}$$

* Basic Rules

$P(A \cap B) = P(A|B)P(B)$

$P(A \cup B) = P(A) + P(B) - P(A \cap B)$

$P(h|D) =$

* Conditional Independence —

↳ X is c.i. of Y, given Z, i.e.

$P(X|Y,Z) = P(X|Z)$, given that:

$P(X,Y|Z) = P(X|Z)P(Y|Z)$

* we want most probable h given D:

$$h_{MAP} = \underset{h \in H}{\operatorname{argmax}} P(h|D)$$

→ when all $h \in H$ equally likely, h_{MAP} reduced to $h_{ML} = \underset{h \in H}{\operatorname{argmax}} P(D|h)$

* expensive to find conditional

instead of what is the h_{MAP}, what is the most probable classification result?

Bayes's Optimal Classifier:

↳ if a new instance can take classification $c_j \in C$, then probability of correctly classifying as c_j is:

$$P(c_j|D) = \sum_{h \in H} \underbrace{P(c_j|h)}_{\text{class given } h} \underbrace{P(h|D)}_{h \text{ given } D}$$

→ optimal is argmax

→ assumes: $P(c|D) = P(c|h)P(h|D) + P(c|\neg h)P(\neg h|D)$

Naive Bayes:

SCHEAP when assuming $P(a_1 \dots a_n|c) = \prod_i P(a_i|c)$ i.e. c.i. to ↓ dim

* take # possible instances of possible target vals

given attribute values $\langle a_1, \dots a_n \rangle$, give classification $c \in C$ s.t.

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(c_j|a_1, \dots a_n) = \underset{c_j \in C}{\operatorname{argmax}} P(a_1, \dots a_n|c) P(c)$$

estimate from D

→ easy — learning

Intro

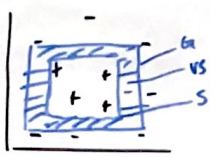
supervised	unsupervised	reinforcement
$\{x_1 \dots x_n \to y\}$	$\{x_1 \dots x_n\}$	$s_{t+1} = \delta(s_t, a_t)$ $r_{t+1} = p(s_t, a_t)$

Challenges:
- dimensionality ↑ → ↑ sample size, generalization, sample complexity
- discover structure, features, redundancy, hyperparameters, metrics
- discrete states, scalability, assumptions

Supervised Learning Theory

↳ Version Space: $\{h \in \mathcal{H} \mid h \in [S, G]\}$

↳ most & least specific h



↳ Sample Complexity: # examples required to learn a target concept

↳ Error: for $h \in \mathcal{H}$, target c —

$r_D(h) \leftarrow |h(x) \neq c(x)|$

↳ Exhausting $VS_{H,D}$:

ε-exhausted iff —

$\forall h \in VS_{H,D} \mid error_D(h) < \varepsilon$

$error_D(h)$ can be bounded given and if $|H|$ is finite, D has $m \geq 1$ iid. examples of some target c, then $P(\text{not } \varepsilon\text{-exhausted } VS_{H,D}) < |\mathcal{H}| e^{-\varepsilon m}$

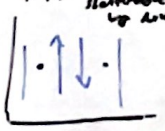
and if $P(\text{"}) \leq \delta$ desired,

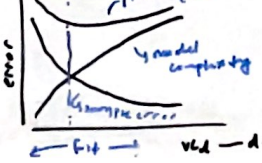
$m \geq \frac{1}{\varepsilon}(\ln|H| + \ln(1/\delta))$

what if $|H| \to \infty$?

use VC dimension: max # m shattered by $h \in H$ (linear classifier VCD = d+1)

4 points not shattered by lines





$* \uparrow VC(H) \leftrightarrow \uparrow m$

$m \geq \frac{1}{\varepsilon}\left(4\lg(2/\delta) + 8VC(H)\lg\left(\frac{13}{\varepsilon}\right)\right)$

VCd — directly related to sample complexity

Hidden Units & Layers

↳ considering tanh, sigmoid

each layer $f(Wx + b)$ consists —

1. linear transformation by W
2. translation by b
3. pointwise activation by f

↳ each layer can stretch/squeeze space, but cannot change topology

i.e. if $\det(W) \neq 0$, $N \times N$ layer represents a homeomorphism

$* \det(W) = 0 \to$ collapse one dimension

$* 2$ units → learn a 1D boundary

3 units → learn a planar (2D) boundary

⋮ → hyperplanar boundary

↳ Adding more layers → learn more complicated composition of functions → more curves in boundary ← projection to higher space to find useful topology

Training Rules

→ perceptron rule (linearly separable, small η)

$w_i \leftarrow w_i + \eta(t - o)x_i$

↳ delta rule / gradient descent (multi-best approx.)

$\Delta w_i \leftarrow \Delta w_i + \alpha(t - o)x_i$, $\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$

$w_i \leftarrow w_i + \Delta w_i$

for $\sigma(z)$ — ↳ $\sigma(E(w))(1 - \sigma(E(w)))$

of FFN Power: 2 layer → any boolean or continuous bounded function (linear output)

3 → arbitrary function

Reinforcement Learning

↳ DCV: $V^\pi(s_t) = r_t + \lambda r_{t+1} + \lambda^2 r_{t+2} + \dots = \sum_0^\infty \lambda^i r_{t+i}$

→ discount; 0 → current reward, → 1 → future rewards

↳ find optimal policy to maximize $V^*(s)$

↳ given reward, state transition, and eval function, optimal policy

has same values for all actions equidistant from goal state

$\hat{Q}(s,a) \leftarrow \hat{r} + \lambda \max_{a'} \hat{Q}(s', a')$

(reward, not exact from s, a to s')

$(\leftarrow r(s,a) + \lambda V^*(\delta(s,a)))$

↳ guaranteed to converge iff:

1) deterministic MDP?

2) bounded reward i.e. $(\forall s, a) \mid r(s,a)| < c$

3) all (s,a) pairs visited inf

↳ what if ND MDP?

$V^\pi(s) = E(r_t)$,

$Q(s,a) = E(r_t)$

$= E(r) + \lambda \sum_{s'} T(s') \dots$

$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \lambda Q(s',a') - Q(s,a)]$

of learned w/a decaying weighted average

↳ $\alpha_n = \frac{1}{1 + visits}$